

RAP Cloud 課程 2：Managed BDEF——CUD 真正端對端執行

Lecture

這一課要證明的事情

舊課程 rap03 完整教了 Managed BDEF 語法 (`managed;`、`create;`/`update;`/`delete;`)，但因為 On-Premise 系統的 `CL_CSP_MD_METADATA_FACTORY` 白名單機制，CUD 執行到底層一律 Dump (`.claude/rules/sap-adt-mcp.md` 第 43 節) ——語法教得再完整，學員從來沒機會親眼看到 Managed CUD 真正跑完一次。

這一課只有一個目標：讓你親眼看到 **Managed CUD** 真的端對端跑成功。已經用 ABAP Unit 完整驗證：CREATE → 讀回確認 → UPDATE → 讀回確認 → DELETE → 讀回確認，全部通過，沒有任何 Dump。

Managed BDEF：最終能動的版本

```
managed;

define behavior for ZI_RC01_TASK alias Task
persistent table zrc01_task
lock master
etag master created_at
{
    create;
    update;
    delete;

    field ( readonly : update ) task_id;
    field ( mandatory )          description;
}
```

跟舊課程 rap03 的版本相比，唯一新語法是 `field ( readonly : update )`——這個「限定操作範圍」寫法（只在 Update 時唯讀，Create 時仍可設定）舊課程 rap09 實測過 **On-Premise 系統不支援**（會報 `" | , " expected, not ":"` 語法錯誤），這個 **Cloud 環境** 確認可以用，這是這一課第一個具體驗證到的「版本落差」證據。`task_id` 是主鍵，這門課沒有宣告自動編號（Early/Late Numbering，留到 rc05），所以 client 必須在 CREATE 時自己指定 `task_id`，但建立後不該再被改——`field(readonly:update)` 精準表達了這個語意，比舊課程用過的「整欄位唯讀」更準確。

Eclipse ADT Step by Step：建立空殼（本課實際操作記錄）

1. 對著 `ZI_RC01_TASK` 這個 CDS View 右鍵，找「New Behavior Definition」（若沒有直接選項，走 New → Other ABAP Repository Object → 搜尋 **Behavior Definition**）。
2. 精靈會要求選 **Implementation Type**，選 **Managed**（這一步官方精靈就是在問你 BDEF 要走 Managed 還是 Unmanaged 實作模式，這門課全程用 Managed）。
3. Name 自動帶出跟 CDS View 同名，Package `ZRAPCLOUD`，Finish。

4. 精靈產生的骨架不能直接用，需要調整：

```
" Eclipse 精靈自動產生的骨架 ( 不能直接用，見下方說明 )
managed implementation in class zbp_i_rc01_task unique;
strict ( 2 );

define behavior for ZI_RC01_TASK //alias <alias_name>
persistent table zrc01_task
lock master
authorization master ( instance )
//etab master <field_name>
{
  create ( authorization : global );
  update;
  delete;
  field ( readonly ) task_id;
}
```

精靈預設就幫你寫好 **strict(2)** 跟 **authorization master(instance)**——代表 SAP 官方認為這是新建 Managed BDEF 該有的標準配置，不是我們自己加的。但這一課刻意簡化掉這兩樣東西，過程跟原因見下一節。

為什麼刻意拿掉精靈自動生成的 **strict(2)** 跟 Implementation Class——用真實錯誤訊息追出的因果鏈

直接照抄精靈骨架的 **strict(2)** + **authorization master(instance)** 去啟用，會依序踩到兩個錯誤：

第一步：**managed; strict(2);** (不宣告 **authorization**)，啟用報：

```
ERROR: The behavior definition is "strict", which means that every entity
      must be flagged as "authorization master" or as "authorization dependent".
```

第二步：補上 **authorization master (global)**，啟用報：

```
ERROR: Operations need to be implemented for the entity "ZI_RC01_TASK",
      which means an implementation class needs to be specified.
```

這條因果鏈說明了一件事：**strict(2)** 本身沒有直接要求 **Implementation Class**，是它強制要求宣告 **authorization**，而宣告 **authorization master** 才連帶要求要有 **Implementation Class** 去實作檢查邏輯。反過來說：

BDEF 寫法	需不需要 Implementation Class
managed; (純 CRUD，不加 strict ，不宣告 authorization)	❌ 完全不需要 ——這一課實際用的版本，框架全包，你不用寫任何一行 ABAP Class

```
managed; strict(2); + authorization
master(...)
```

✅ 一定需要——必須實作授權檢查邏輯

這一課的選擇：先用零 Implementation Class 的版本，把「Managed CUD 真的能跑」這件事單獨驗證清楚，不要一次引入太多新東西。`strict(2)` 與 Implementation Class 留到 rc03 (Determination / Validation) 一起做——rc03 本來就一定要寫 Handler Method、一定要有 Implementation Class，那時候把 `strict(2)` 也一併打開最省事，不用現在先做一次、rc03 再重做一次。

`strict(2)` 到底是什麼？數字 2 又代表什麼？（查證官方文件，不是猜的）

查證來源：[ABENRAP_STRICT_MODE_GLOSRY](#) / [ABENBDL_STRICT_1](#) / [ABENBDL_STRICT_2](#) (ABAP Keyword Documentation · Cloud 版)。

strict 是什麼（官方原文翻譯）：BDEF strict mode 對 RAP Behavior Definition 套用額外的語法檢查——確保不使用過時語法、隱含可用的操作要明確宣告出來、RAP BO 符合最佳實踐。**BDEF strict mode** 會持續推出更嚴格的版本，每個新版本套用更多額外檢查。

數字 2 是「版本號」，不是「嚴重程度分級」——這是最容易誤解的地方，數字代表 strict mode 檢查規則集的版本，不是「1 級警告、2 級錯誤」這種嚴重度概念：

寫法	版本	涵蓋範圍
<code>strict;</code> (不帶數字)	Version 1	基礎的 strict mode 檢查集
<code>strict(2);</code>	Version 2	套用 Version 1 的所有檢查，再疊加額外的檢查——官方原文：「Strict mode version 2 applies all checks from strict mode version 1, plus some additional syntax checks.」

Version 2 具體多檢查什麼（官方原文）：「Strict mode version 2 introduces mainly stricter checks for the RAP BO contract. Contract violations lead to runtime errors, instead of failed assertions.」——白話翻譯：Version 2 主要是針對「RAP BO Contract」（RAP 業務物件對外承諾的行為規範，例如這一課踩到的「宣告了就要真的實作」）加嚴檢查，違反這些規則的後果從「執行期斷言失敗」升級成「執行期錯誤」——這代表 Version 2 是站在「早點在開發階段抓到問題，別等執行期才爆炸」的立場，把原本比較寬鬆的檢查收緊。另外官方也提到：`strict(2)` 是把 RAP BO 發布成「C0 Contract」（一種可擴充性等級）API 的前提條件——如果你的 RAP BO 之後要開放給別人擴充（例如透過 BDEF Extension），沒有 `strict(2)` 是不夠格的。

這一課實測到的 `strict(2)` 行為，正好呼應官方說法：`authorization` 宣告在沒有 `strict` 的普通 Managed BDEF 裡是選擇性的（可以不宣告，框架允許），但在 `strict(2)` 之下變成強制——這正是官方說的「RAP BO Contract」的一部分：`strict(2)` 認為「這個 BO 到底要不要做權限檢查」是一個不能含糊帶過的契約項目，逼你明確表態，這也是為什麼在沒有先準備好 Implementation Class 的情況下，貿然打開 `strict(2)` 反而會卡關。

這一課的 ABAP Unit 測試：EML 逐步排錯記錄（值得完整看一次）

寫測試類別的過程中，連續踩到三個 EML 語法坑，逐一用 `get_abap_diagnostics` 精準定位、查證官方文件修正，過程本身很有參考價值：

① **COMMIT ENTITIES** 要接 **FAILED/REPORTED**，前面必須先加 **RESPONSE OF <entity>**：

" ❌ 錯誤：直接接 **FAILED**，報「Unable to interpret "FAILED"」

```
COMMIT ENTITIES
  FAILED DATA(ls_failed)
  REPORTED DATA(ls_reported).
```

" ✅ 正確：要先宣告 **RESPONSE OF** 是哪個 Entity

```
COMMIT ENTITIES
  RESPONSE OF zi_rc01_task
  FAILED DATA(ls_failed)
  REPORTED DATA(ls_reported).
```

(如果完全不需要知道成功/失敗細節，也可以只寫最短形式 **COMMIT ENTITIES.**，靠 **sy-subrc** 判斷成敗即可——這個短版本不需要 **RESPONSE OF**。)

② **DELETE** 操作只能用 **FROM**，不能用 **WITH**——官方文件原文：「The **FROM** variant is the only option for **DELETE** and **EXECUTE** in most cases.」：

" ❌ 錯誤：報「"FROM" expected after "DELETE"」

```
MODIFY ENTITIES OF zi_rc01_task ENTITY Task
  DELETE
    WITH VALUE #( ( %key-task_id = lv_id ) )
  ...
```

" ✅ 正確

```
MODIFY ENTITIES OF zi_rc01_task ENTITY Task
  DELETE
    FROM VALUE #( ( %key-task_id = lv_id ) )
  ...
```

CREATE / UPDATE 用 **WITH** (因為要傳實際欄位值)，**DELETE / EXECUTE** (呼叫 Action) 用 **FROM** (因為只需要指定要動哪一筆，不需要傳欄位值)——這是這兩組關鍵字的分工原則。

③ **BDEF** 裡 **task_id** 標成完全 **readonly**，會連 **CREATE** 都不能設定 (本課上一節已經講過，這裡是它在 EML 層面的具體症狀)：**CREATE FIELDS (...)** 清單裡放了一個完全唯讀的欄位，會報 **The field "TASK_ID" of entity "... cannot be modified.**，而且會**連鎖**讓後面好幾行程式碼的錯誤訊息變得莫名其妙 (**Field "LS_FAILED_CREATE-TASK" is unknown** 這種)，因為編譯器從第一個錯誤點開始就已經對不上狀態了。**排錯教訓**：EML 陳述式裡如果看到一串「看起來不相關」的連鎖錯誤，先檢查最前面那一句是不是本身就有問題，不要逐一去改後面那些看起來獨立的錯誤。

完整測試方法 (**ZCL_RC02_TASK_TEST=>CREATE_UPDATE_DELETE**)：CREATE 一筆測試資料 (**task_id = 'RC02TEST01'**) → **COMMIT ENTITIES** → **READ ENTITIES** 讀回確認欄位值正確 → **UPDATE** 改 **status** → 讀回確認改成功 → **DELETE** 清掉測試資料 → 讀回確認真的刪除 (**RESULT** 表格 0 筆)。全程用 **cl_abap_unit_assert** 斷言，測試資料用 **RC02TEST** 前綴避免跟這個共用系統上其他人的資料混淆，結束時自己清乾淨。

ABAP Unit 執行結果

```
Unit Test Results for CLAS/I ZCL_RC02_TASK_TEST.main
Status: ALL TESTS PASSED
Total: 1 | Passed: 1 | Failed: 0

[PASS] ZCL_RC02_TASK_TEST
[PASS] CREATE_UPDATE_DELETE (0.490s)
```

這是這門課到目前為止最重要的一次驗收：Managed BDEF 的 CREATE / UPDATE / DELETE，從 EML 呼叫、到框架自動處理持久化、到資料真的寫進/改進/清出資料庫，全程沒有任何 Dump、沒有白名單擋下來——舊課程 rap03 完全做不到這件事。

學習目標

- 能講出這個 Cloud 環境跟舊課程最核心的差異：Managed CUD 真的能執行完，不會被 SAP 白名單擋下來
- 能寫出最基本、不需要 Implementation Class 的 Managed BDEF (純 `managed;` + `create;/update;/delete;`)
- 知道 `field(readonly:update)` 這個限定範圍語法在這個 Cloud 環境可用，跟 On-Premise 系統不支援形成對照
- 能講出「`strict(2)` 強制要求宣告 `authorization`，宣告 `authorization` 才連帶要求 Implementation Class」這條因果鏈，不會誤以為 `strict(2)` 本身直接要求 Implementation Class
- 能正確引用官方文件講出 `strict` / `strict(2)` 的差異：數字代表版本號 (Version 2 = Version 1 的所有檢查 + 額外的 RAP BO Contract 檢查)，不是嚴重程度分級；知道 Contract 違反會從斷言失敗升級成執行期錯誤；知道 `strict(2)` 是發布 C0 Contract 可擴充 API 的前提
- 能分辨 `execute_data_query` (MCP 工具，查資料現況用) 跟 EML (真正的 ABAP 語言，驅動 RAP BO 行為用) 的差異，不會把兩者混為一談
- 能寫出正確的 EML CREATE / UPDATE / DELETE / COMMIT ENTITIES / READ ENTITIES 語法，知道 DELETE 用 `FROM` 不是 `WITH`，`COMMIT ENTITIES` 要有回應時必須先 `RESPONSE OF <entity>`
- 能用 ABAP Unit (`FOR TESTING` / `cl_abap_unit_assert`) 驗證 RAP BO 的完整生命週期，知道這是這個環境對應舊課程 `programrun` 的驗證手段

物件清單

物件	名稱	型別
Managed Behavior Definition	<code>ZI_RC01_TASK</code> (跟 CDS View 同名)	BDEF/BDO
ABAP Unit 測試類別	<code>ZCL_RC02_TASK_TEST</code>	CLAS/OC

套件：`ZRAPCLOUD` (沿用 rc01)。兩個物件都已在 Eclipse 建立空殼、Claude 寫入完整內容並啟用成功，`run_unit_tests` 全部通過。

驗證方式

1. `get_abap_diagnostics` 確認兩個物件都無錯誤 (BDEF 有 1 個「建議加 `strict`」的軟性警告，刻意先不理會，原因見上方說明)

2. `abap_activate` 兩個物件都回報 `Activation successful`
3. `run_unit_tests` 對 `ZCL_RC02_TASK_TEST` 執行，`ALL TESTS PASSED` (見上方完整輸出)

思考題

1. 如果不宣告 `field(mandatory) description`，CREATE 時故意不傳 `description` 欄位，你預期會發生什麼事？(提示：想想這個 `annotation` 的字面意思，這一課的測試沒有驗證這個情境，你可以自己在 Eclipse 試著改一次測試方法看實際行為)
2. `etag master created_at` 這一行如果拿掉，會不會影響這一課的測試結果？(提示：`etag` 的用途是併發修改檢測，這一課的測試是單一序列操作，沒有併發情境)
3. 承上一節「`strict(2)` 因果鏈」——如果你只想要「強制宣告 `authorization`」的檢查，但不想要 Version 2 額外加的其他檢查，可以怎麼做？(提示：回顧 `strict` 不帶數字 vs `strict(2)` 的差異，`strict(2)` 是 `strict` 的超集合)
4. 這一課的 `field(readonly:update)` 舊課程 rap09 證實 On-Premise 系統不支援。如果你手上剛好也有一個支援新版語法的系統，還有沒有其他舊課程 rap09 提到、但這個環境版本應該也支援的語法 (`field(mandatory:create)`、`authorization master(global) / lock:none` 等)？可以自己試著在這個環境驗證看看。

答案

見 `zi_rc01_task.bdef.abap`、`zcl_rc02_task_test.clas.abap`。SAP 端物件：`ZI_RC01_TASK` (Behavior Definition，跟 CDS View 同名不同型別)、`ZCL_RC02_TASK_TEST` (ABAP Unit 測試類別)，套件 `ZRAPCLOUD`，`run_unit_tests` 執行結果：`ALL TESTS PASSED`。